

NIOS-II Custom Instruction

Instructions personnalisables

Instructions personnalisables

Contenu

- Introduction
- Instructions personnalisées sur NIOS II
- Hardware
- Appel via le langage C

Source :

http://www.intel.com/literature/ug/ug_nios2_custom_instruction.pdf

Instructions personnalisables ...

Introduction

Un processeur a **un jeu d'instruction défini lors de sa conception.**

Ce jeu d'instructions est fait pour être le plus efficace dans des cas généraux, mais **pas pour des cas particuliers.**

Les DSP (Digital Signal Processor) ont des instructions spécialisées non adaptées pour des applications à usage général.

Avec certains processeurs, il est possible d'ajouter des instructions au jeu d'instructions initial.

Instructions personnalisables ...

Introduction

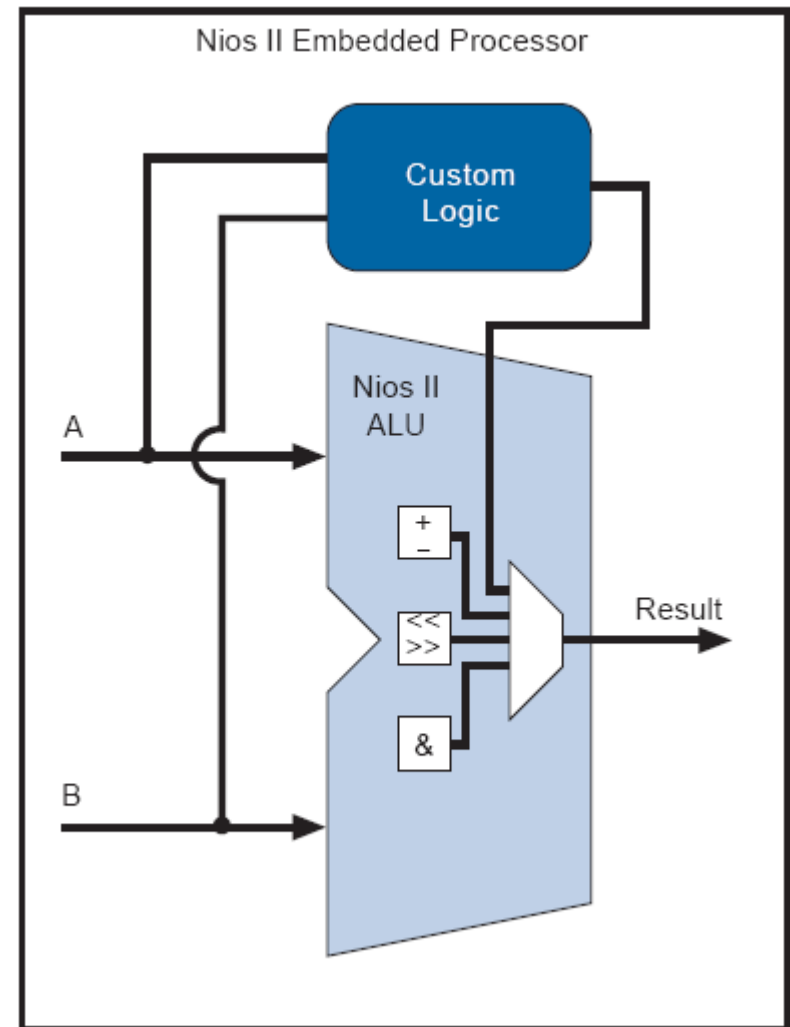
Quelques fois, il est nécessaire d'accélérer le temps de calcul. L'optimisation est possible via l'un des choix suivants :

- la réécriture partielle du code le plus efficacement
- l'écrire une partie en langage assembleur
- l'utilisation d'un meilleur processeur
- l'utilisation d'un **coprocesseur** spécialisé
- l'utilisation de multiprocesseurs
- la translation d'une partie du code vers une implémentation matérielle => **accélérateur sur FPGA par exemple**

Instructions personnalisables ...

Instruction personnalisée avec NIOS (Custom Instruction)

Mise en parallèle
avec l'ALU du NIOS
d'un bloc logique
personnalisé



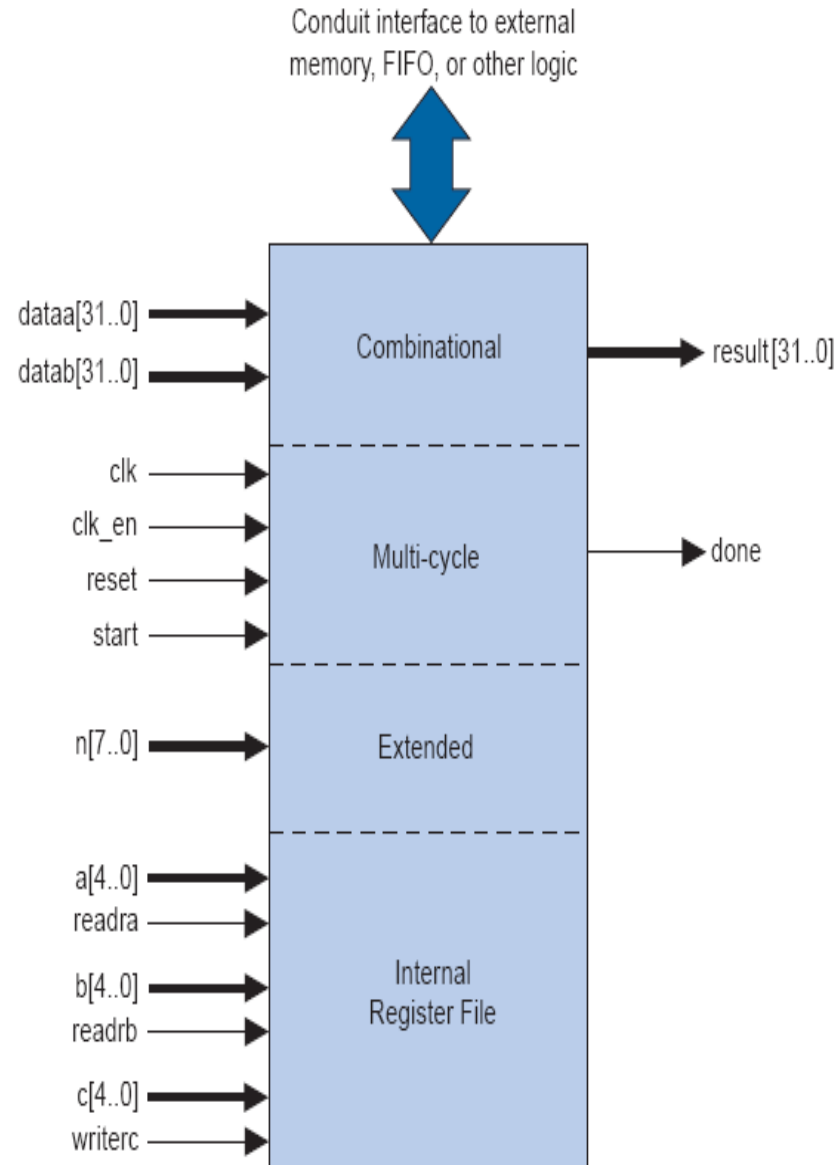
Instructions personnalisables ...

Implémentations possibles

Elle peut être :

- **Combinatoire**
- **Multi cycles**
- **Extended** (jusqu'à 256 instructions)
- Fichier de **Registres**
- **Accès externe**

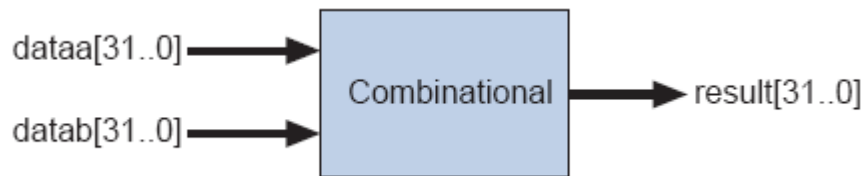
Noter la relation avec le bus Avalon



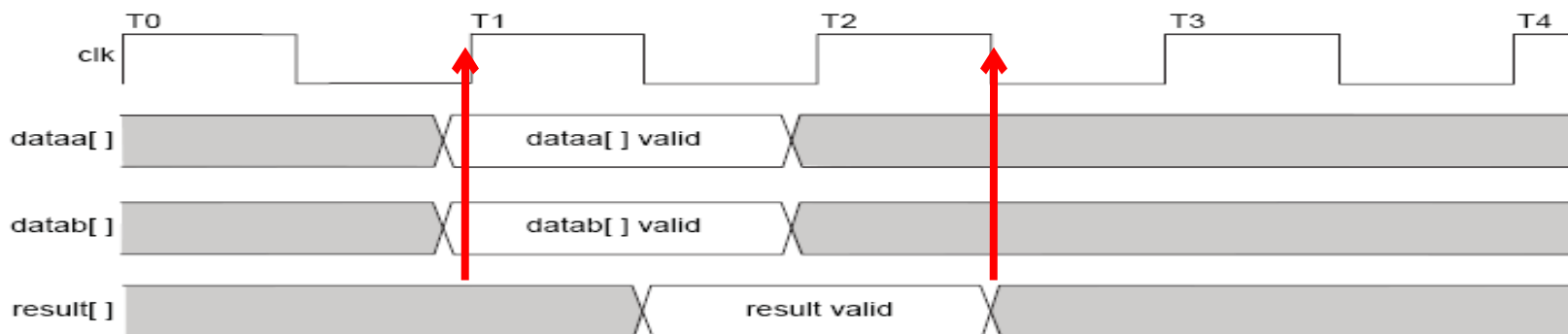
Instructions personnalisables ...

Implémentation combinatoire

- 2 * 32 bits data, 32 bits résultat
- 1 cycle d'horloge



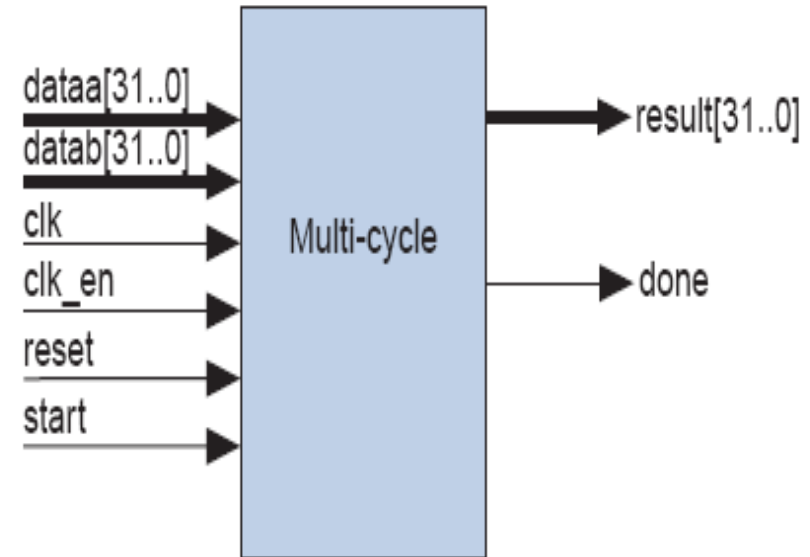
Port Name	Direction	Required	Description
dataa[31:0]	Input	No	Input operand to custom instruction
datab[31:0]	Input	No	Input operand to custom instruction
result[31:0]	Output	Yes	Result of custom instruction



Instructions personnalisables ...

Instruction multi cycles

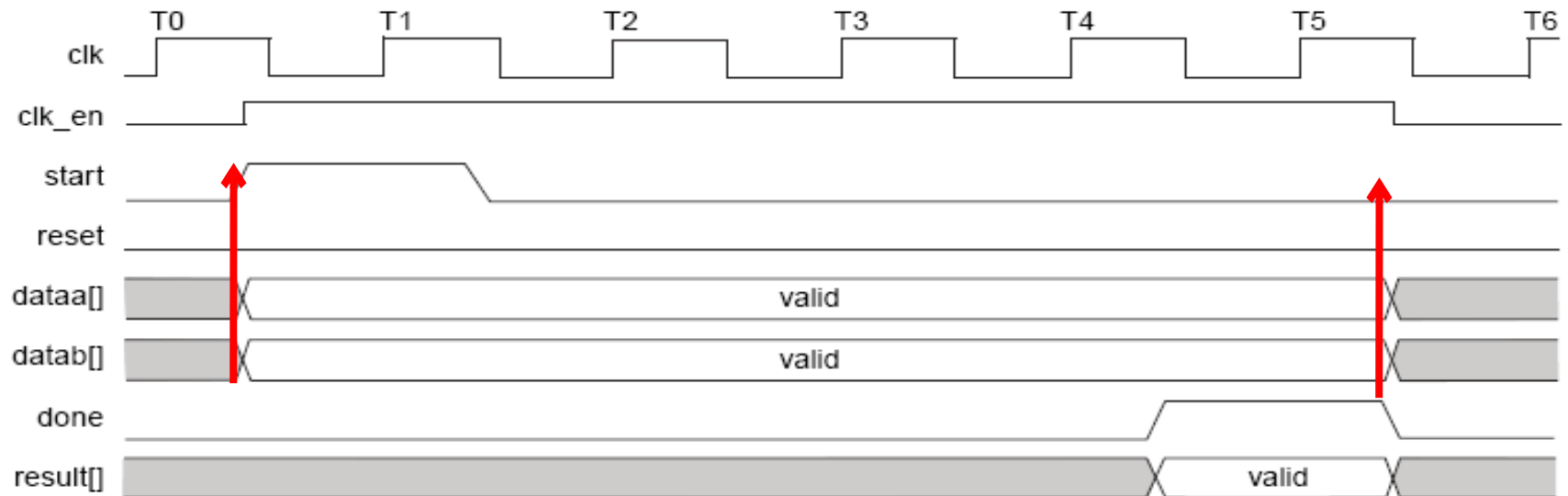
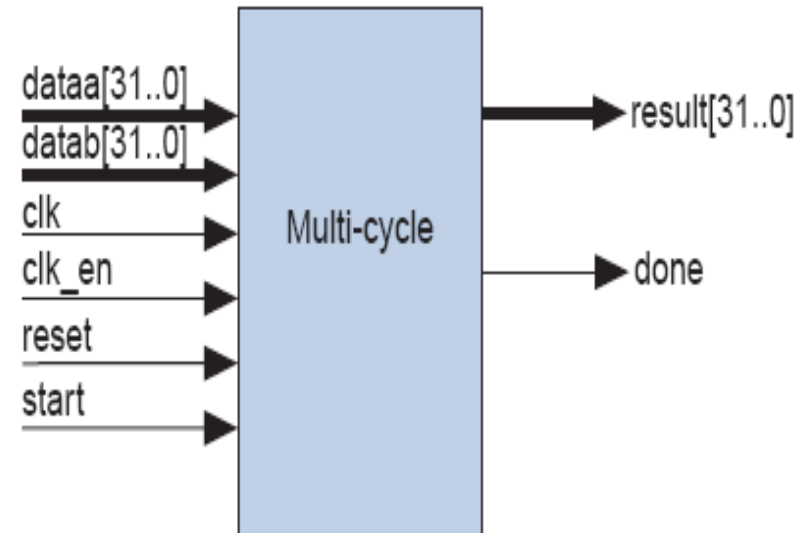
- 2 * 32 bits data, 32 bits résultat
- n cycles d'horloge pour le résultat
- start et done pour le handshaking



Port Name	Direction	Required	Description
clk	Input	Yes	System clock
clk_en	Input	Yes	Clock enable
reset	Input	Yes	Synchronous reset
start	Input	No	Commands custom instruction logic to start execution
done	Output	No	Custom instruction logic indicates to the processor that execution is complete
dataa[31:0]	Input	No	Input operand to custom instruction
datab[31:0]	Input	No	Input operand to custom instruction
result[31:0]	Output	No	Result of custom instruction

Instructions personnalisables ...

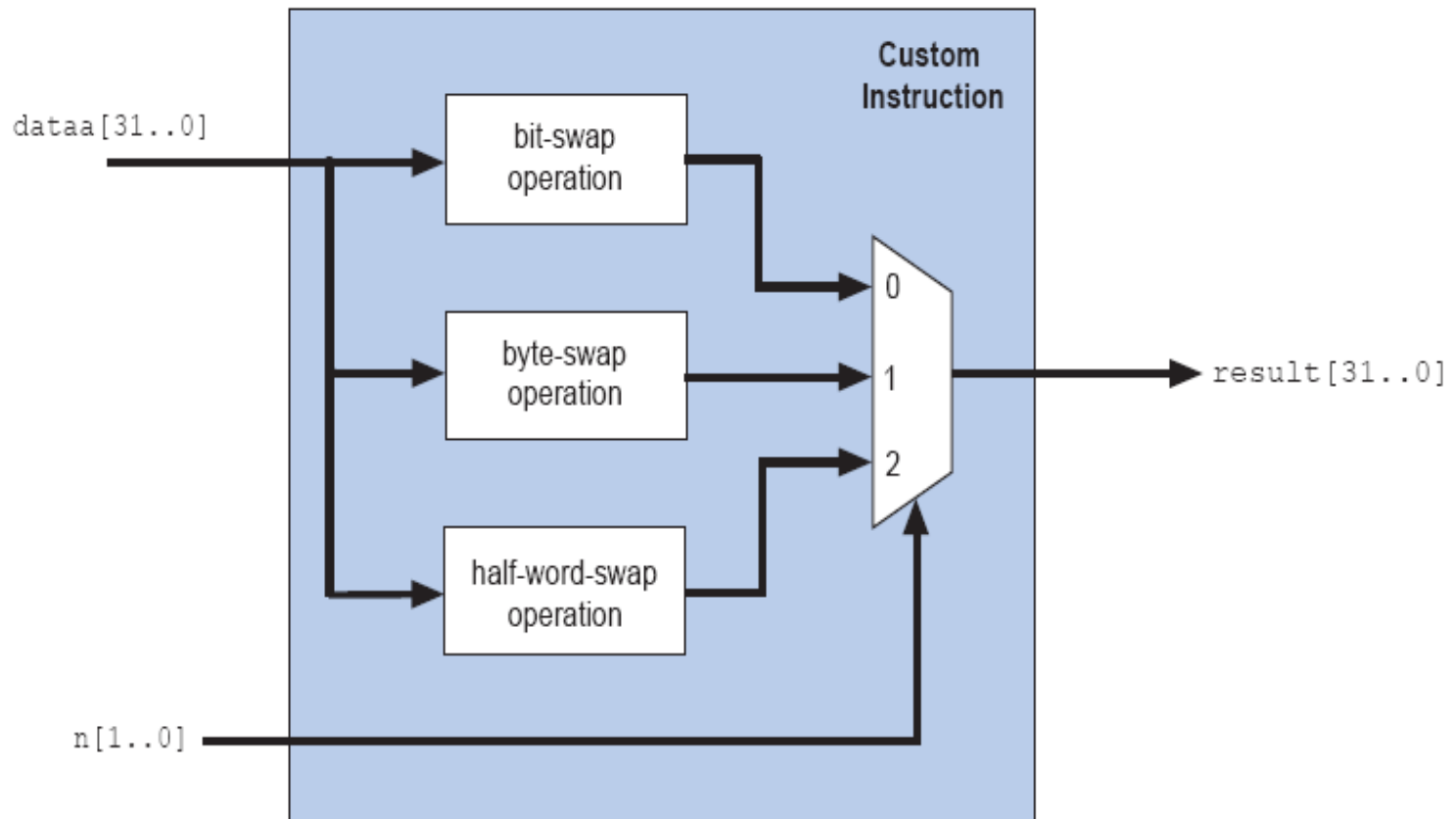
Instruction multi cycles



Instructions personnalisables ...

Instructions étendues

- Plus d'une instruction par bloc
- $n[\dots]$ index d'instruction



Instructions personnalisables ...

Fichier de Registres internes

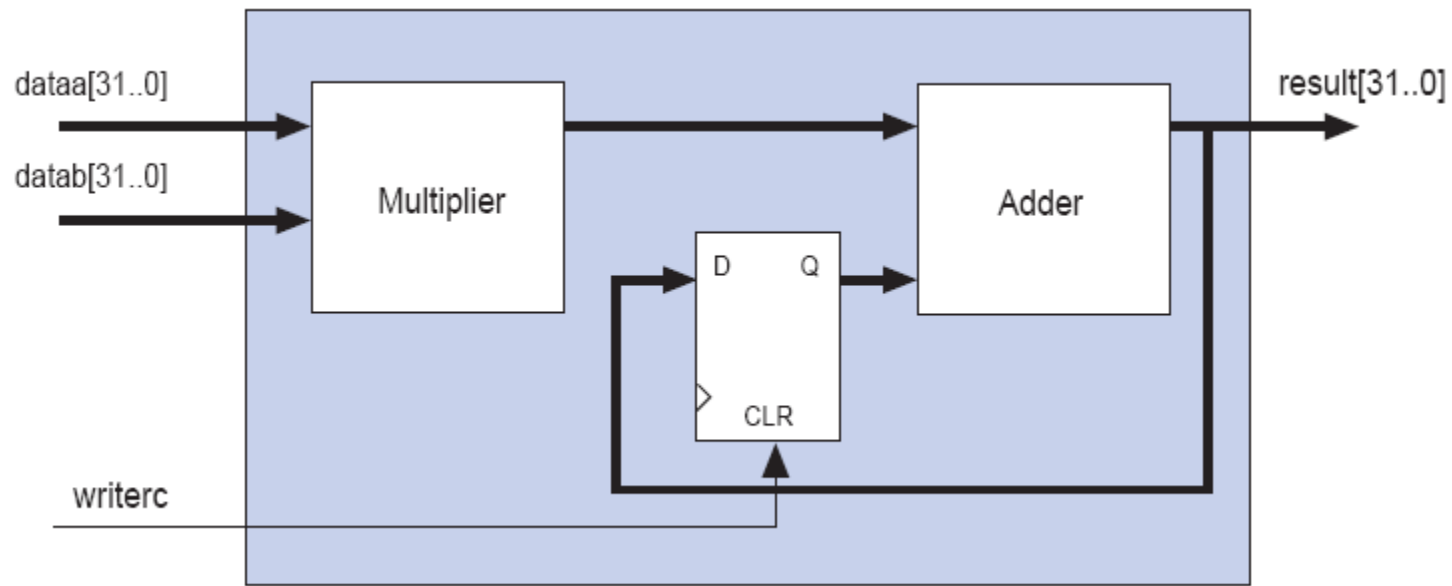
- Registres internes -> jusqu'à 32
- 2 registres de sources (Ra, Rb)
- 1 registre de destination (Rc)
- Peut être mixé avec Dataa, Datab ou Datac
- Bus de données 32 bits

Port Name	Direction	Required	Description
readra	Input	No	If readra is high, the Nios II processor supplies dataa; if readra is low, custom instruction logic reads the internal register file indexed by a.
readrb	Input	No	If readrb is high, the Nios II processor supplies datab; if readrb is low, custom instruction logic reads the internal register file indexed by b.
writerc	Input	No	If writerc is high, the Nios II processor writes to the result port; if writerc is low, custom instruction logic writes to the internal register file indexed by c.
a[4:0]	Input	No	Custom instruction internal register file index
b[4:0]	Input	No	Custom instruction internal register file index
c[4:0]	Input	No	Custom instruction internal register file index

Instructions personnalisables ...

Fichier de Registres internes

- Exemple de sélection mixte de données
`dataa`, `datab` and `WriteRc`

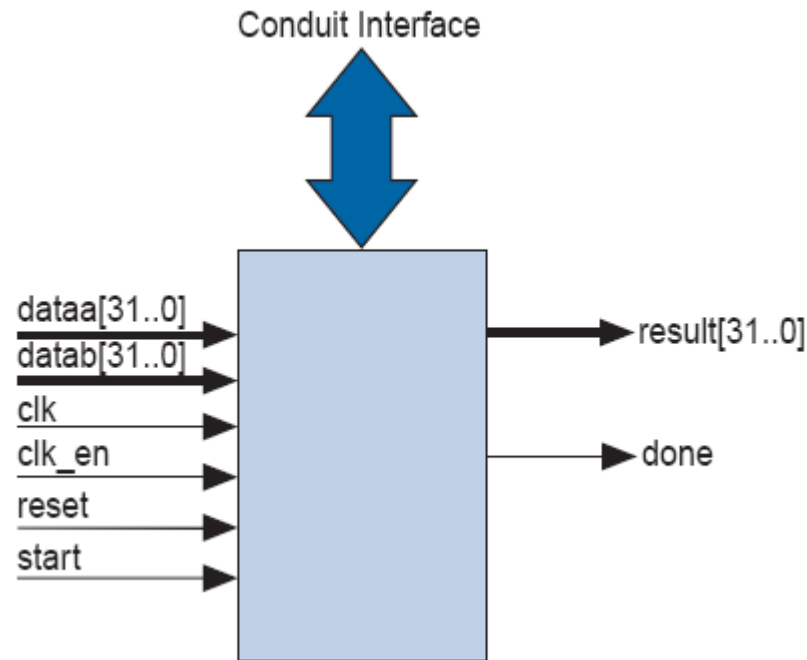


Cellule **Mac** (une des caractéristiques d'un DSP)

Instructions personnalisables ...

Interface externe (conduit)

- Accès externe alloué aux accès multi-cycle
- Disponible également pour le fichier de registres internes

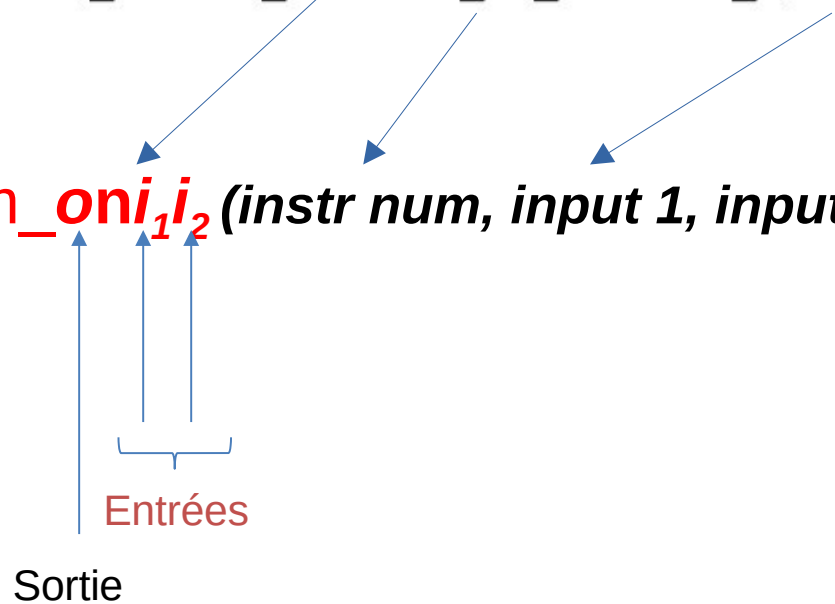


Instructions personnalisables ...

- Comment est appelée l'instruction personnalisée en langage C

```
#define ALT_CI_BITSWAP_N 0x00  
#define ALT_CI_BITSWAP(A) __builtin_custom_ini(ALT_CI_BITSWAP_N, (A))
```

- Utilise l'extension de gcc
- Instruction: `__builtin_custom_oni12(instr num, input 1, input 2)`
 - Types of *o*, *i*₁ et *i*₂:
 - i integer
 - f float
 - p void *



Instructions personnalisables ...

- Exemple:

- *void *_ builtin_custom_***pnif** (*int n, int dataa, float datab*);

- **pnif** :

- *p* output: void *
- *n* séparateur
- *i* input 1: integer
- *f* input 2: float (32 bits)

- Les 3 paramètres ***o, i₁ et i₂*** ***ne sont pas tous obligatoires***

```
void _ builtin_custom_nf (int n, float dataa);  
float _ builtin_custom_fnpi (int n, void * dataa, int  
datab);
```

← **Aucune sortie**

Instructions personnalisables ...

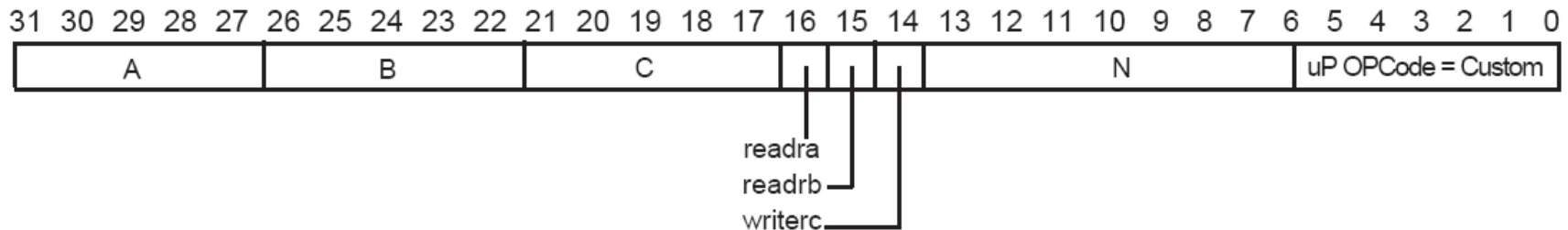
Exemple d'utilisation

- 2 Instructions définies :

```
1. /* define void undef_macro1(float data); */
2. #define UDEF_MACRO1_N 0x00
3. #define UDEF_MACRO1(A) __builtin_custom_nf(UDEF_MACRO1_N, (A));
4. /* define float undef_macro2(void *data); */
5. #define UDEF_MACRO2_N 0x01
6. #define UDEF_MACRO2(B) __builtin_custom_fnp(UDEF_MACRO2_N, (B));
7.
8. int main (void)
9. {
10.     float a = 1.789;
11.     float b = 0.0;
12.     float *pt_a = &a;
13.
14.     UDEF_MACRO1(a);
15.     b = UDEF_MACRO2((void *)pt_a);
16.     return 0;
17. }
```

Instructions personnalisables ...

En langage assembleur



Instruction Fields: A = Register index of operand A

B = Register index of operand B

C = Register index of operand C

N = 8-bit number that selects instruction

readra = 1 if instruction uses processor's register rA, 0 otherwise

readrb = 1 if instruction uses processor's register rB, 0 otherwise

writerc = 1 if instruction provides result for processor's register rC, 0 otherwise

- Custom opcode : 0x32

Instructions personnalisables ...

Instruction assembleur

- Syntaxe Assembleur :
 - custom N, xC, xA, xB
 - N: Numéro d'instruction depuis ***system.h***
 - x:r Registres NIOS II vers dataa, datab et resultat. readra, readrb à **1**
 - x: c Output writerc at **1**

Instructions personnalisables ...

Implémentation

- Implémentation en HDL (VHDL) à l'aide de Qsys (Platform designer)

Exemple

Implémentation d'une fonction combinatoire XOR.
Inclure un timer configuré en **mode Time Stamp**

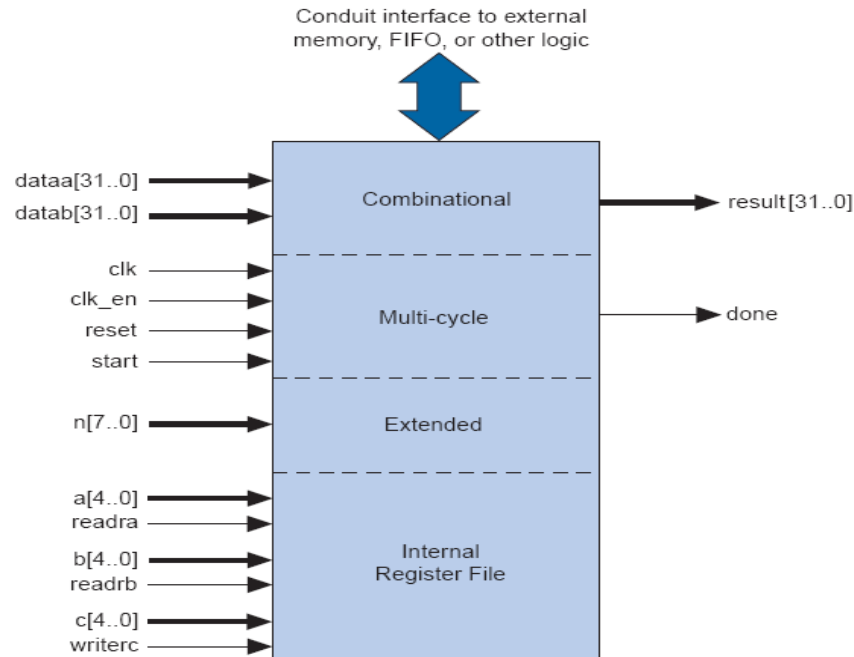
Instructions personnalisables

Hardware

Sachant une implémentation d'un circuit combinatoire, on doit considérer, au plus, les signaux Dataa, dataB et Result (Figure ci-bas)

On doit considérer deux fichiers VHDL

- Un **fichier d'interface** entre le NIOS et le circuit réalisant la fonction cible
- Un **fichier décrivant le composant implémentant la fonction cible**



Fichier HDL d'interface

```
library ieee;  
use ieee.std_logic_1164.all;
```

```
entity customcombinationalInterface is
```

```
port(  
    signal n: in std_logic_vector(2 downto 0);  
        signal dataa: in std_logic_vector(31 downto 0);  
        signal datab: in std_logic_vector(31 downto 0);  
        signal result: out std_logic_vector(31 downto 0)  
);
```

```
end entity customcombinationalInterface;
```

```
architecture a_customcombinationalInterface of customcombinationalInterface is
```

```
COMPONENT myxor is port(  
    signal s: in std_logic_vector(2 downto 0);  
    signal a: in std_logic_vector(31 downto 0);  
    signal b: in std_logic_vector(31 downto 0);  
    signal r: out std_logic_vector(31 downto 0)  
);
```

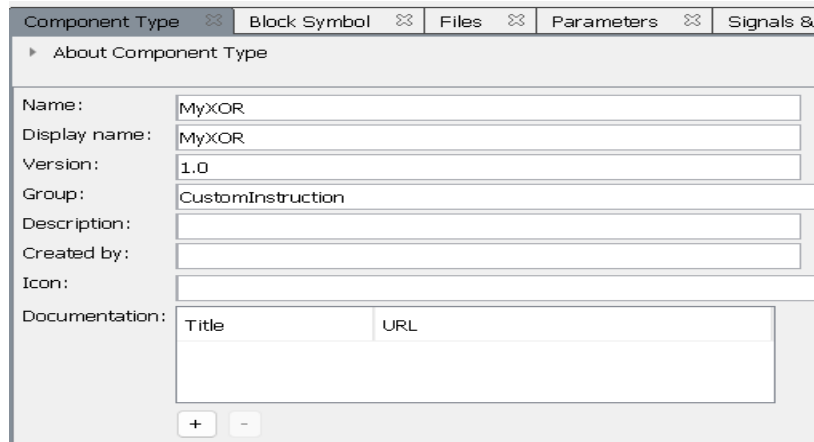
```
END COMPONENT;
```

```
begin  
    customcombinational_instance: myxor PORT MAP (  
        n,  
        dataa,  
        datab,  
        result);
```

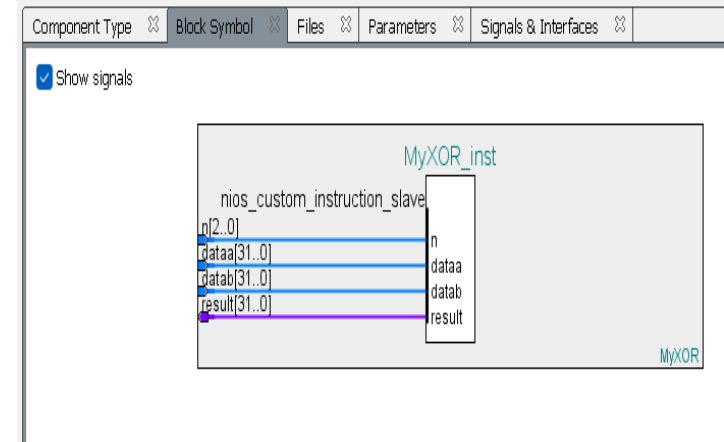
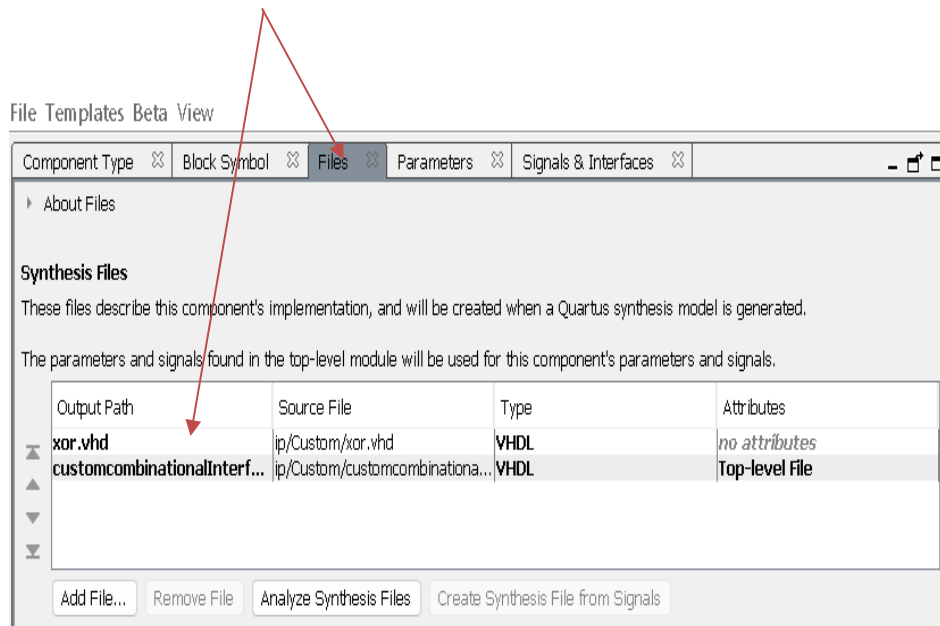
```
end architecture a_customcombinationalInterface;
```

```
end architecture a myxor;
```

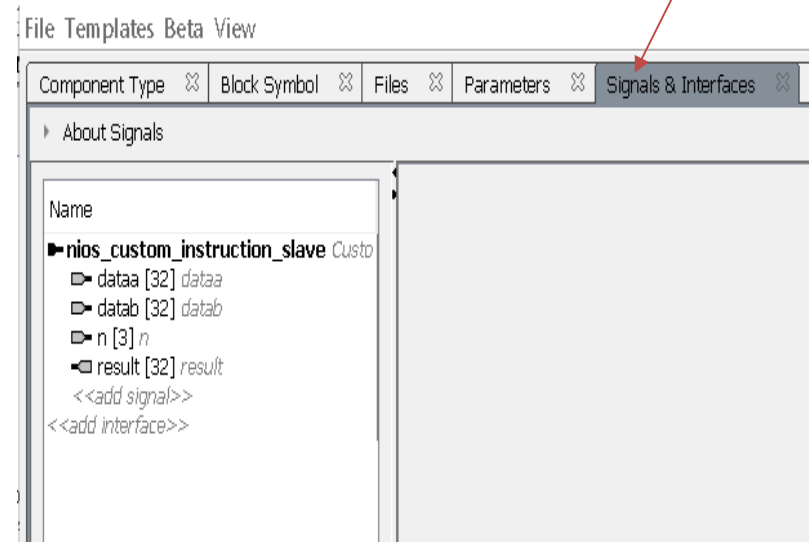
Création du composant sous QSYS Designer



Fichiers de description



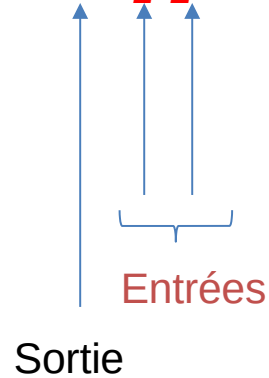
Signaux et **interface**



Software

Utilisation de l'instruction

builtin_custom_oni₁i₂ (*instr num, input 1, input 2*)



Avec :

- i : integer
- f : float
- p : void *

Générée et configurée automatiquement et localisée sur le BSP (fichier system.h)

```
/*
```

```
* Custom instruction macros
```

```
*
```

```
*/
```

```
#define ALT_CI_MYXOR_0(n,A,B) __builtin_custom_inii(ALT_CI_MYXOR_0_N+(n&ALT_CI_MYXOR_0_N_MASK),(A),(B))
```

```
#define ALT_CI_MYXOR_0_N 0x0
```

```
#define ALT_CI_MYXOR_0_N_MASK ((1<<3)-1) ← on impose 8 instructions au plus pour MYXOR  
en évitant tout débordement
```

Programme principale avec mesure de temps

```
if (alt_timestamp_start() < 0)
{
    printf ("No timestamp device available\n");
}
else
{
    while(indx<100)           // Calcul via une instruction spécialisée
    {
        Ticks_time1[indx] = alt_timestamp();
        c= ALT_CI_MYXOR_0(0,a,b);
        Ticks_time2[indx++] = alt_timestamp();
    }
    indx = 0;
    while(indx<100)           // Calcul via une opération native
    {
        Ticks_time11[indx] = alt_timestamp();
        c= a ^ b;
        Ticks_time22[indx++] = alt_timestamp();
    }
}
```

Exercise

- Pour tester les capacités de l'implémentation logicielle vs matérielle, concevoir un système NIOSII pour réaliser des fonctions binaires :
 - A 32 bits input a31..a0
 - A 32 bits output result o31..o0
- sachant les 3 cas suivants

A31.. A24	-> o7 .. o0	Changement de position de l'octet <code>result(31 downto 24) <= dataa(7 downto 0);</code>
A23 .. A8	-> o8 .. o23	Changement de l'ordre des bits <code>gen_m : for i in 0 to 15 generate result(8 + i) <= dataa(23 - i); end generate gen_m;</code>
A7 .. A0	-> o31.. o24	Changement de position de l'octet <code>result(7 downto 0) <= dataa(31 downto 24);</code>

Exercise

- Exécuter ces fonctions en langage C à l'aide d'opérations natives
- L'implémenter en tant qu'instruction personnalisée
- Mesurer les temps d'exécution dans les deux cas et tirer une conclusion
- Modifier, appliquer et évaluant sachant une table de 1000 valeurs